

Evolutionary Database Design

Incremental improvements to the database schema so that it can be continuously updated with changes, reflecting the customer's requirements.

What EDD is about?

Is **not** a database design pattern

Is about how to maintain a database when the system is continuously **evolving**.

EDD is a **design process**, not the design itself.

Database Design

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Non-destructive

```
ALTER TABLE users ADD COLUMN location VARCHAR(100);
```

- location is **nullable** (no impact on old data or existing queries)
- Old code continues to work as-is
- New code can check for null or use the new column if available

Destructive

```
ALTER TABLE users RENAME COLUMN name TO full_name;
```

- Existing code expecting the name column will **break**
- Queries, APIs, and UI layers relying on name will fail

Handling the Destructive Change Safely

Phase 1: Start (Non-breaking)

```
-- Add the new column
ALTER TABLE users ADD COLUMN full_name VARCHAR(255);
-- Backfill it
UPDATE users SET full_name = name;
-- Ensure new code starts writing to both
```

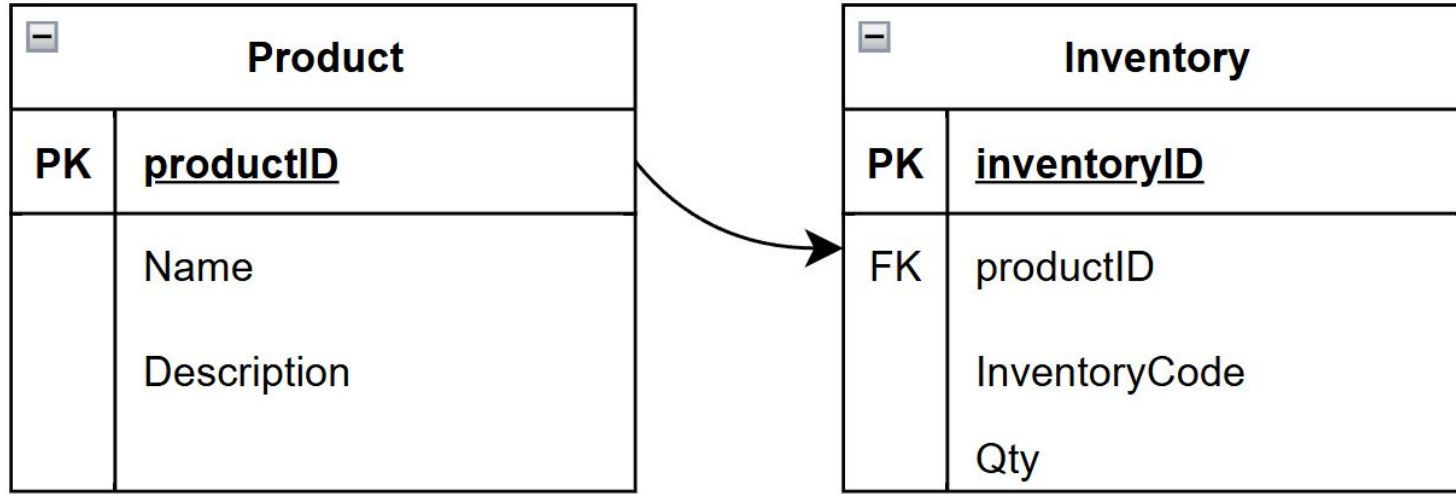
Phase 2: Transition

Application writes to both **name** and **full_name**
Reads can fall back: *COALESCE*(full_name, name) [returns the **first non-null value**]

id	name	full_name
1	Alice	NULL
2	Bob	Bob Johnson

Phase 3: End (Safe deletion)

```
-- Once all clients use `full_name`, drop the old column
ALTER TABLE users DROP COLUMN name;
```



Updated User Story: user should be able to *see, search, and update* the location, batch, and serial numbers of a product in inventory.

Currently, InventoryCode holds these values (e.g. CTG_023_768)

Solution: split the field and store in three fields

How to do this?

1. Writes a SQL migration script to change the schema and also migrate all the existing data in the inventory.
2. Run it locally.
3. Change the application code to use the new columns.

```
ALTER TABLE inventory ADD location_code VARCHAR2(6) NULL;  
ALTER TABLE inventory ADD batch_number VARCHAR2(6) NULL;  
ALTER TABLE inventory ADD serial_number VARCHAR2(10) NULL;  
  
UPDATE inventory SET location_code = SUBSTR(product_inventory_code,1,6);  
UPDATE inventory SET batch_number = SUBSTR(product_inventory_code,7,6);  
UPDATE inventory SET serial_number = SUBSTR(product_inventory_code,11,10);  
  
DROP INDEX uidx_inventory_code;  
  
CREATE UNIQUE INDEX uidx_inventory_identifier  
  ON inventory (location_code,batch_number,serial_number);  
  
ALTER TABLE product_inventory DROP COLUMN inventory_code;
```

How to do this?

4. Change any database code, such as views, stored procedures and triggers, to use the new columns
5. Change any indexes based on the inventory_code column
6. Modify/Run the test suits.
7. Commit this database migration script and all the application code changes to the version control system

DBAs collaborate closely with developers

WHY? The **developer** knows what new functionality is needed & **DBA** has a global view of the data in the application and other surrounding applications.

All database artifacts are version controlled with application code in the same repository that's being used by everyone else.

WHY? There's only one place to look. Easy to audit if any problems occur. Easy to handle out-of-sync database.

All database changes are migrations

Previously... developers would change the database as per their need using schema editing tools and ad-hoc SQL for standing data, then DBA would compare the development database with production database and update the changes. Here, the purpose of the changes needs be understood again, by a different group of people.

Now... every change is written in a migration script which is version controlled together with application code changes.

All database changes are migrations....

Migration script-

- Writes SQL into a text file inside a migrations folder within the project's version control repository
- Add unique IDs to each migration (look up the highest currently used number in the migrations folder, and uses that number together with a description to name the file.)
- Track which migrations were applied and their sequence using a **ChangeLog** table.

All database changes are migrations....

```
select * from changelog;
```

24/24Ln. 1 Col. 25Lines: 1INSERTWRITABLE \n UTF8Delimiter: ;

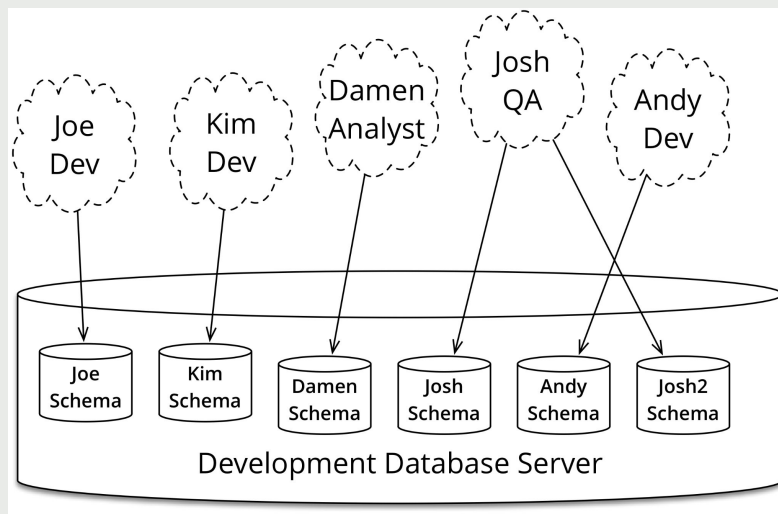
☒ changelog

OFF

	CHANGE_NUMBER	COMPLETE_DT	APPLIED_BY	DESCRIPTION	
1	1	2016-08-04 12:12:13.475	jen_dev	1_asset.sql	
2	2	2016-08-04 12:12:13.525	jen_dev	2_asset_type.sql	
3	3	2016-08-04 12:12:13.57	jen_dev	3_asset_parameters.sql	
4	4	2016-08-04 12:12:13.586	jen_dev	4_inventory.sql	
5	5	2016-08-04 12:12:13.628	jen_dev	5_split_inventory_code.sql	
6	6	2016-08-04 12:12:13.671	jen_dev	6_equipment_type.sql	
7	7	2016-08-04 12:12:13.717	jen_dev	7_add_insurance_value_to_equipment_type.sql	
8	8	2016-08-04 12:12:13.723	jen_dev	8_data_location_equipment_type.sql	

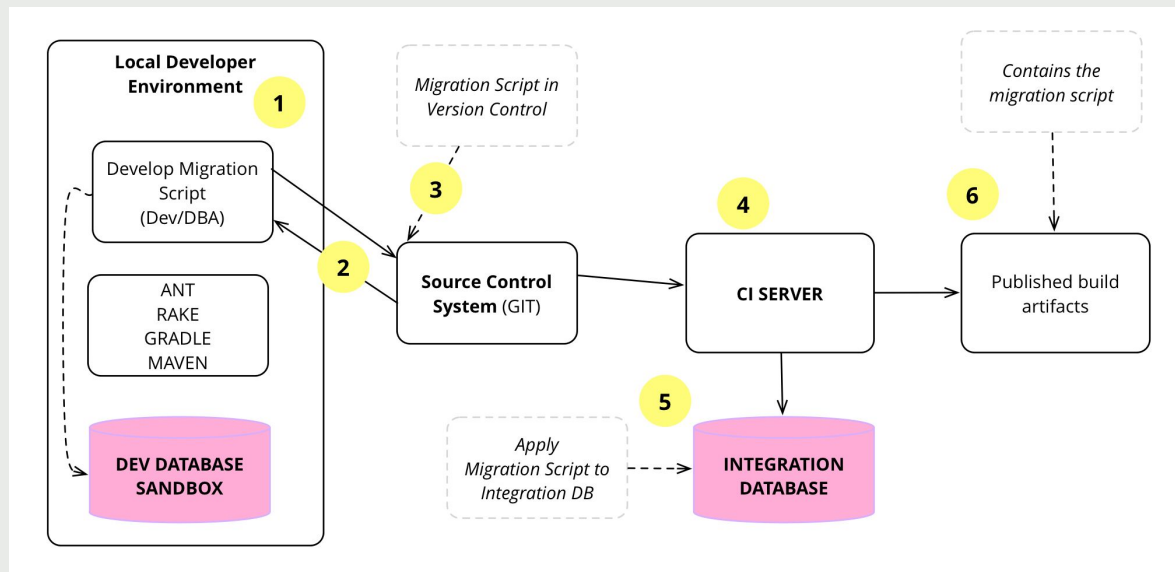
Everybody gets their own database instance

WHY? to enable isolated, uninterrupted experimentation—just like working with private code copies. This prevents conflicts from half-done changes in a shared database.



Developers continuously integrate database changes

WHY? to avoid complex, error-prone merges by catching and resolving conflicts early through frequent, small updates.



Database design example...

A database consists of schema and data

Not only code but also data- i.e. Static data (e.g. countries) or Sample data for testing (mostly fictional but can be real data from legacy system).

All database changes are refactorings

One of the big differences about database refactorings is that they involve **three** different changes that have to be done together-

- a. Changing the database schema
- b. Migrating the data in the database
- c. Changing the database access code

Thus whenever we describe a database refactoring, we have to describe all three aspects of the change and ensure that all three are applied before we apply any other refactorings.

Automate the refactorings

Frameworks like Liquibase and Active Record Migrations provide a DSL to apply database refactorings, allowing a standard way to apply database migrations.

Developers can update their database on-demand

Clearly separate all DB access code (separate database layer)

It reduces the need for developers to have deep SQL knowledge, simplifying their work. For DBAs, it offers a clear view of how the database is being modified, aiding in performance tuning and optimization.

Release frequently

References-

<https://martinfowler.com/articles/evodb.html>
